

Relaxing a truth table: what is settled and what is not

Digital Defiance — HELUT project

August 2026

Abstract

A lookup table in a digital circuit is a list of bits. Let those bits slide continuously in $[0, 1]$ and a circuit becomes a smooth function you can optimise over, exact at the corners of the cube. This note states the objective we use, the elementary facts we have checked about it, the classical result that explains why driving the penalty up terminates on real gates, and the two questions we have left. It is self-contained: no familiarity with the surrounding project is assumed and nothing here requires special hardware. Everything below is reproducible with stdlib Python 3 in a couple of seconds.

1 A motivating example, and why it is honest

Two toy block ciphers, built on one skeleton: 16-bit block, 16-bit key, four rounds, four-bit S-boxes, a bit permutation between rounds, round keys by rotation. Identical wiring, identical schedule. The only difference is the S-box. One is the PRESENT S-box [1], which is differentially 4-uniform. The other is an invertible affine map over $\text{GF}(2)$, so every output bit is a parity of input bits. Printed side by side the two S-boxes look equally scrambled.

Measurement	Nonlinear S-box	Affine S-box
S-box differential uniformity (max of DDT)	4/16	16/16
Random triples failing $E(x) \oplus E(y) \oplus E(z) = E(x \oplus y \oplus z)$	200/200	0/200
Best 4-round differential over the full codebook	2^{-10}	2^0
Plaintexts recovered from one known pair, zero key search	0/64	64/64
Truth-table constants that are affine in their inputs	0/4	4/4

Table 1: Both ciphers are weak; only one is structurally dead. An ideal 16-bit permutation would give roughly 2^{-16} in row three. Differential counts are exact over all 2^{16} plaintexts, restricted to one- and two-bit input differences. Reproduce with `python3 Scripts/toy_cipher_demo.py`.

Neither cipher is any good, and the comparison is deliberately between weak and structurally dead rather than between strong and weak. A 16-bit key falls to exhaustive search immediately, and four rounds is far too few. None of the attacks in the table are new: differential and linear cryptanalysis are due to Biham and Shamir [2] and Matsui [3].

The last row is the one that motivates the rest of this note. Once the circuit is synthesised, each S-box output bit is a four-input truth table: a 16-bit constant. The structural defect is visible in those constants, at the level below the cipher. That is the level we would like to optimise over.

2 The relaxation

Fix a circuit of L lookup tables, each with k inputs, so each table is specified by 2^k bits. Replace the bits by real numbers. A table with entries $u \in [0, 1]^{2^k}$ evaluated at input $x \in \{0, 1\}^k$ becomes

the multilinear extension

$$\tilde{u}(x) = \sum_{j \in \{0,1\}^k} u_j \prod_{i=1}^k [x_i j_i + (1 - x_i)(1 - j_i)], \quad (1)$$

which agrees with the original table whenever u is binary, and interpolates otherwise. Composing tables along the circuit's wiring gives a map $y : [0, 1]^{2^k L} \rightarrow \mathbb{R}^m$, where m counts observed output bits over a fixed set of input patterns.

Write $w \in [0, 1]^n$, $n = 2^k L$, for the concatenated genome. Given a target $t \in \mathbb{R}^m$, set

$$F_{\text{data}}(w) = -\|y(w) - t\|_2^2, \quad (2)$$

$$\pi(w) = \sum_{i=1}^n w_i(1 - w_i), \quad (3)$$

$$F_\lambda(w) = F_{\text{data}}(w) - \lambda \pi(w), \quad \lambda \geq 0. \quad (4)$$

Search over F_λ with λ increasing on a schedule, then round each coordinate to a bit. The penalty π vanishes exactly on $\{0, 1\}^n$, so raising λ is meant to drive the solution to a genuine circuit.

3 What is settled

The following are elementary and machine-checked in the accompanying repository. We record them not because they are difficult but because they are the complete list of what the construction is entitled to assume.

Proposition 1 (Structure of the objective). *For F_λ as in (4) with $\lambda \geq 0$:*

1. $\pi \geq 0$ on $[0, 1]^n$, with $\pi(w) = 0$ if and only if $w \in \{0, 1\}^n$;
2. $F_{\text{data}} \leq 0$, with equality if and only if $y(w) = t$;
3. for fixed w , the map $\lambda \mapsto F_\lambda(w)$ is nonincreasing, and strictly decreasing when $\pi(w) > 0$;
4. the coordinatewise rounding $E(w)_i = \mathbf{1}[w_i \geq \frac{1}{2}]$ is the identity on $\{0, 1\}^n$;
5. a set of disjoint transpositions on a finite alphabet is an involution, and remains one under the mutation operator used here;
6. coordinates held frozen contribute nothing to π .

Clause 4 deserves emphasis because it is weaker than it looks. It says rounding fixes points already at the corners. It does *not* say that rounding a fractional w improves F_{data} , since rounding moves $y(w)$ too.

The one case where the picture is complete is the separable one.

Proposition 2 (Separable case). *Suppose $y(w) = w$, meaning a single fully observed table with no composition, and let $t \in \{0, 1\}^n$. Then for every $\lambda \geq 0$, $w = t$ is the unique global maximiser of F_λ on $[0, 1]^n$, and $F_\lambda(t) = 0$.*

This is immediate: the objective separates into coordinates $f(w_i) = -(w_i - t_i)^2 - \lambda w_i(1 - w_i)$, and for $t_i = 0$ one has $f(w_i) = w_i[(\lambda - 1)w_i - \lambda] < 0$ for all $w_i \in (0, 1]$, with the symmetric statement at $t_i = 1$.

4 What the λ schedule is actually doing

Proposition 2 is the case where the relaxation is trivially exact. Real circuits compose tables, and then y is multilinear in each block separately but not jointly, and F_{data} is neither concave nor convex. The question we started with was whether raising λ genuinely forces the optimiser onto real gates, or whether we were assuming it.

It does, and this is not new. Maximising F_λ is the same as minimising

$$f(w) + \lambda \pi(w), \quad f(w) = \|y(w) - t\|_2^2 \geq 0, \quad (5)$$

and (5) is the classical *concave exact penalty* for 0–1 programming: π is nonnegative, vanishes exactly on the binary points, and is concave. Raghavachari [4] established the equivalence with the integer problem for a sufficiently large penalty in the linearly constrained case; Giannessi and Niccolucci [5] extended it to nonlinear integer programs; Kalantari and Rosen [6] gave a lower bound on the penalty and showed it cannot be reduced in general. Recent treatments are in Rinaldi [7] and Lucidi and Rinaldi [8]. We record the specialisation we rely on, with its two-line argument, purely so the constant is explicit.

Proposition 3 (Threshold, standard). *Let $f \in C^2([0, 1]^n)$ and $\lambda \geq \frac{1}{2} \sup_{w \in [0, 1]^n} \lambda_{\max}(\nabla^2 f(w))$. Then $f + \lambda \pi$ is concave on $[0, 1]^n$, so it attains its minimum at a vertex; if the inequality is strict the minimisers are vertices only. Since π vanishes there, every such minimiser is a binary genome, and is a minimiser of f over $\{0, 1\}^n$.*

Proof. $\nabla^2 \pi = -2I$, so $\nabla^2(f + \lambda \pi) = \nabla^2 f - 2\lambda I \preceq 0$ under the stated bound, giving concavity. A concave function on a polytope attains its minimum at an extreme point; under strict concavity no point in the relative interior of a positive-dimensional face can be a minimiser, since it is the midpoint of two others. The extreme points of $[0, 1]^n$ are $\{0, 1\}^n$, where $\pi = 0$. $\square$

For $f = \|y(w) - t\|^2$ one has $\nabla^2 f = 2(J^\top J + \sum_k r_k \nabla^2 r_k)$ with $r = y(w) - t$ and J the Jacobian of y , so the threshold is controlled by the conditioning of y and the residual, both bounded on the cube.

The bound is worst-case over the whole cube, so the honest question is how loose it is. On two composed 2-input tables with a target unreachable by any binary genome, we measure

$$\sup_{[0, 1]^8} \lambda_{\max}(\nabla^2 f) = 4 + 2\sqrt{3}, \quad \text{so } \lambda \geq 2 + \sqrt{3} \approx 3.732 \text{ suffices,}$$

attained at a vertex. Directly maximising F_λ on the same topology moves the maximiser from the interior to a vertex somewhere in $(2, 4]$:

λ	$\max F_\lambda$	maximiser
0.00	−0.000	interior
0.25	−0.125	interior
1.00	−0.500	interior
2.00	−1.000	interior
4.00	−1.000	vertex
8.00	−1.000	vertex
16.00	−1.000	vertex

Table 2: Eight sliders, four observed patterns, target $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2})$, which none of the 2^8 binary genomes reaches. Maximisation is coordinate ascent with random restarts, so the F column is a lower bound on the true maximum and the crossover is empirical rather than certified. Reproduce with `python3 Scripts/lambda_threshold_probe.py`; the Hessian bound with `python3 Scripts/penalty_threshold.py`.

So on this topology the classical sufficient bound $2 + \sqrt{3}$ lands inside the bracket where vertices actually take over, which is about as tight as a worst-case bound can be expected to be. That is one eight-dimensional example and we would not generalise from it.

5 What we would still like to know

Two things, both narrower than the question we first wrote down.

Question 1. *How does the threshold of Proposition 3 scale with circuit structure — depth, fan-out, the number of observed patterns — for y built by composing multilinear tables? A bound in terms of the topology rather than a numerically evaluated supremum would say whether practical λ schedules ever actually reach it, or whether they are running below the guarantee and succeeding for a different reason.*

Question 2. *Suppose some binary w^* satisfies $y(w^*) = t$. Above the threshold the minimisers of (5) are vertices, but is every such vertex in the fibre $\{w \in \{0, 1\}^n : y(w) = t\}$? Uniqueness is clearly too much to ask, since distinct binary genomes can agree on every observed pattern; the question is whether the recovered vertex fits the data at all, or whether the penalty can prefer a worse-fitting one.*

Question 2 has a straightforward affirmative answer whenever the threshold in Proposition 3 holds, since the minimiser of $f + \lambda\pi$ over the cube then minimises f over $\{0, 1\}^n$ and $f(w^*) = 0$ is the global minimum. What we do not know is what happens in the regime a real schedule occupies, below the sufficient bound, where the objective is not yet concave.

6 What this note does not claim

The relaxation is a search heuristic. Nothing here shows that optimising F_λ recovers a key, breaks a cipher, or succeeds on an arbitrary netlist. The affine S-box above is caught by a linearity test that predates this project by three decades; the relaxation is not what found it. Propositions 1 and 2 are checked by executable property tests over exhaustive small domains and seeded pseudorandom sampling, not by a proof assistant, and formalising them properly is open work. Proposition 3 is not ours: it is the standard exact-penalty argument, cited above, and we state it only to fix the constant. The measured $4 + 2\sqrt{3}$ is a numerical supremum over a finite sample of one eight-dimensional cube, not a proof about that topology.

7 Reproducing

```
python3 Scripts/toy_cipher_demo.py           # the two ciphers, Table 1
python3 Scripts/tensorlut_math_ref.py        # Propositions 1 and 2
python3 Scripts/lambda_threshold_probe.py    # Table 2, the observed crossover
python3 Scripts/penalty_threshold.py         # the Hessian bound,  $2 + \sqrt{3}$ 
```

Stdlib only, Python 3.8 or newer, no build step and no particular hardware. The wider project this comes from is a Swift and Metal prototype that evaluates gate-level netlists under homomorphic encryption, and that part does require Apple Silicon; none of the mathematics above does. Entry point: `INTRO.md` in the repository.

References

- [1] A. Bogdanov et al. *PRESENT: An Ultra-Lightweight Block Cipher*. CHES 2007.

- [2] E. Biham and A. Shamir. *Differential Cryptanalysis of DES-like Cryptosystems*. CRYPTO 1990.
- [3] M. Matsui. *Linear Cryptanalysis Method for DES Cipher*. EUROCRYPT 1993.
- [4] M. Raghavachari. *On Connections Between Zero-One Integer Programming and Concave Programming Under Linear Constraints*. Operations Research 17(4):680–684, 1969.
- [5] F. Giannessi and F. Niccolucci. *Connections between nonlinear and integer programming problems*. Symposia Mathematica 19:161–176, 1976.
- [6] B. Kalantari and J. B. Rosen. *Penalty for zero-one integer equivalent problem*. Mathematical Programming 24:229–232, 1982.
- [7] F. Rinaldi. *New results on the equivalence between zero-one programming and continuous concave programming*. Optimization Letters 3:377–386, 2009.
- [8] S. Lucidi and F. Rinaldi. *Exact Penalty Functions for Nonlinear Integer Programming Problems*. Journal of Optimization Theory and Applications 145:479–488, 2010.